Gold Price Prediction Using Machine Learning

Nurmukhammed Baltabayev
Nurbay Kengessov
Assel Yessenzholkyzy
Kirgizbayev Yernar
Madina Dyussebayeva

Supervisor: Zhaniya Medeuova

Contents

1 Topic and Motivation

1.1 Project Topic

The main objective of this project is to predict gold prices using machine learning techniques based on historical financial market data.

Gold price forecasting is an important problem in the financial domain because gold is considered one of the most valuable and stable investment assets in the world. Investors, financial institutions, and analysts continuously monitor gold prices to make informed investment and trading decisions.

In this project, historical gold price data containing features such as Open, High, Low, Close, and Volume was used to develop predictive machine learning models. Both regression and classification approaches were explored to analyze future price movements and market direction.

1.2 Motivation

Accurate gold price prediction can provide valuable support for investors, traders, and financial analysts. Since gold prices are influenced by economic conditions, inflation, geopolitical events, and market uncertainty, forecasting future prices is a challenging task.

Traditional statistical methods often struggle to capture complex nonlinear relationships and temporal patterns present in financial time-series data. Machine learning models can automatically learn hidden patterns from historical data and improve prediction performance.

The motivation of this project is to explore how different machine learning algorithms perform on gold price prediction tasks and identify the most effective model for forecasting future market behavior.

Additionally, this project aims to demonstrate the importance of data preprocessing, feature engineering, and model evaluation in building reliable machine learning systems for financial forecasting.

2 Dataset and Preparation

2.1 Dataset Description

The dataset used in this project contains historical gold price data collected from Kaggle. The dataset includes daily market information such as opening price, closing price, highest price, lowest price, and trading volume.

The dataset covers multiple years of gold trading activity, allowing the analysis of long-term trends and financial market behavior.

The main dataset features are presented in Table 1.

Feature	Description
Date	Trading date
Open	Opening gold price
High	Highest daily price
Low	Lowest daily price
Close	Closing gold price
Volume	Trading volume

Table 1: Dataset Features

2.2 Data Collection

The dataset was downloaded from the Kaggle platform in CSV format and imported into a Jupyter Notebook environment using Python libraries such as pandas, NumPy, matplotlib, and scikit-learn.

The dataset was loaded and analyzed to understand its structure, variable types, and overall quality before performing machine learning experiments.

2.3 Data Cleaning

Before model training, the dataset was cleaned and preprocessed to improve data quality and ensure reliable model performance.

The following preprocessing steps were performed:

- Checking for missing values
- Removing duplicate rows
- Converting the Date column into datetime format
- Sorting records chronologically
- Removing rows with invalid values

Missing values generated during feature engineering and rolling window calculations were removed using the dropna() function.

2.4 Feature Engineering

Feature engineering was performed to create additional variables that capture market trends, volatility, and historical price behavior.

Several new features were generated from the original dataset:

- Price_Range
- Open_Close_Diff
- High_Low_Ratio
- Daily_Return
- Close_Lag_1
- Close_Lag_3
- Close_Lag_7
- MA_7
- MA_14
- MA_30
- Volatility_7
- Volatility_14
- Year
- Month
- Day
- DayOfWeek
- Target_Next_Close

These engineered features helped machine learning models better understand temporal patterns and market dynamics.

2.5 Data Preparation for Machine Learning

The preprocessing and feature engineering pipeline used in this project is shown in Figure 5.

```python
df["Price_Range"] = df["High"] - df["Low"]
df["Open_Close_Diff"] = df["Close"] - df["Open"]
df["High_Low_Ratio"] = df["High"] / df["Low"]

df["Daily_Return"] = df["Close"].pct_change()

df["Close_Lag_1"] = df["Close"].shift(1)
df["Close_Lag_3"] = df["Close"].shift(3)
df["Close_Lag_7"] = df["Close"].shift(7)

df["MA_7"] = df["Close"].rolling(window=7).mean()
df["MA_14"] = df["Close"].rolling(window=14).mean()
df["MA_30"] = df["Close"].rolling(window=30).mean()

df["Volatility_7"] = df["Daily_Return"].rolling(window=7).std()
df["Volatility_14"] = df["Daily_Return"].rolling(window=14).std()

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["DayOfWeek"] = df["Date"].dt.dayofweek

df["Target_Next_Close"] = df["Close"].shift(-1)

df = df.dropna().reset_index(drop=True)
```

Figure 1: Feature Engineering and Data Preparation Process

The preprocessing pipeline included the creation of lag-based variables, moving averages, rolling volatility features, and date-related variables extracted from the Date column.

Lag features were used to capture previous market behavior, while moving averages and volatility indicators helped identify long-term trends and short-term market fluctuations.

After preprocessing, rows containing missing values caused by lag operations and rolling calculations were removed. The final cleaned dataset was then used for machine learning experiments and model evaluation.

3 Exploratory Data Analysis

3.1 Overview of EDA

Exploratory Data Analysis (EDA) was conducted to better understand the structure of the dataset, identify trends, analyze relationships between variables, and examine historical gold price behavior before training machine learning models.

Several visualizations were used to explore important market patterns and temporal relationships in the data.

3.2 Gold Price Trend and Moving Average Analysis

A line plot and moving average visualization were used to analyze long-term market trends and smooth short-term price fluctuations.

Figure 2: Historical Gold Price Trend

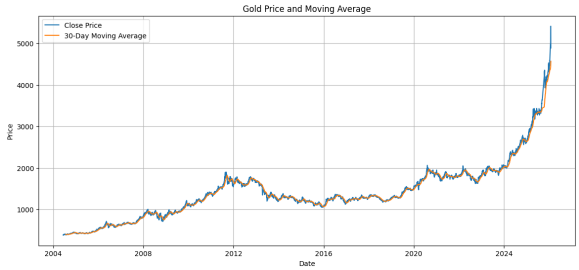

Figure 3: Moving Average Analysis

The visualizations demonstrate a long-term upward trend in gold prices. The moving average smooths market volatility and provides a clearer representation of the overall market direction.

3.3 Correlation Analysis

A correlation heatmap was generated to analyze relationships between numerical variables.

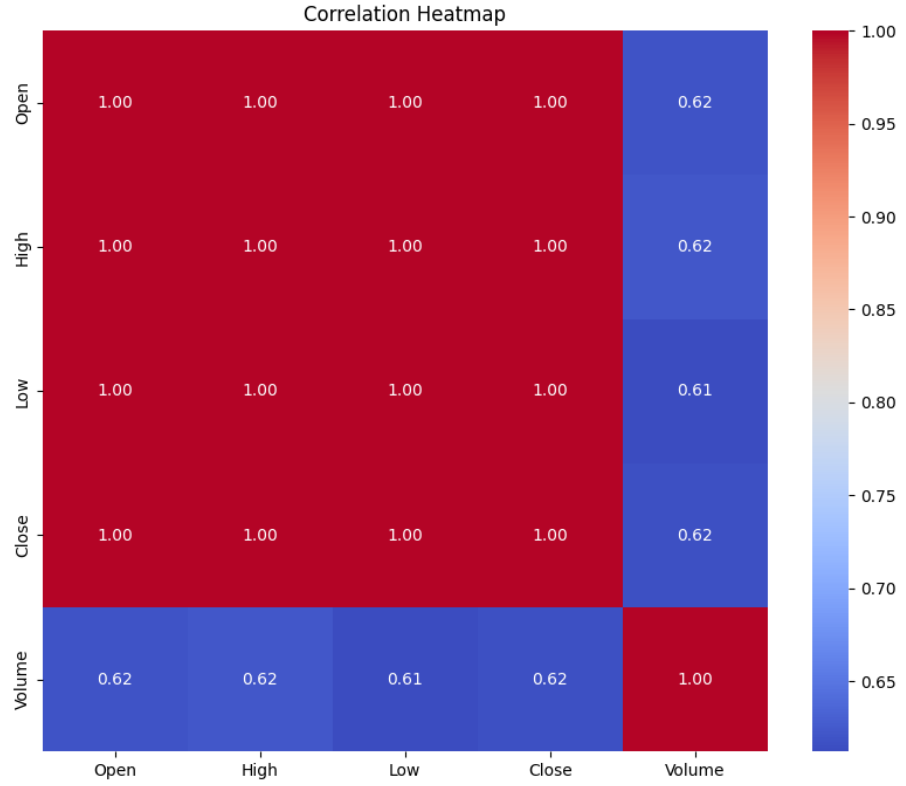

Figure 4: Correlation Heatmap

The heatmap demonstrates a very strong positive correlation between Open, High, Low, and Close prices. This indicates that these variables contain highly related market information and are important for forecasting tasks.

3.4 Feature Engineering Visualization

The preprocessing and feature engineering pipeline used in this project is shown in Figure 5.

```
df["Price_Range"] = df["High"] - df["Low"]
df["Open_Close_Diff"] = df["Close"] - df["Open"]
df["High_Low_Ratio"] = df["High"] / df["Low"]

df["Daily_Return"] = df["Close"].pct_change()

df["Close_Lag_1"] = df["Close"].shift(1)
df["Close_Lag_3"] = df["Close"].shift(3)
df["Close_Lag_7"] = df["Close"].shift(7)

df["MA_7"] = df["Close"].rolling(window=7).mean()
df["MA_14"] = df["Close"].rolling(window=14).mean()
df["MA_30"] = df["Close"].rolling(window=30).mean()

df["Volatility_7"] = df["Daily_Return"].rolling(window=7).std()
df["Volatility_14"] = df["Daily_Return"].rolling(window=14).std()

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["DayOfWeek"] = df["Date"].dt.dayofweek

df["Target_Next_Close"] = df["Close"].shift(-1)

df = df.dropna().reset_index(drop=True)
```

Figure 5: Feature Engineering and Data Preparation Process

Several additional variables were generated during preprocessing, including lag-based features, moving averages, volatility indicators, and date-related variables.

These engineered features helped machine learning models better capture temporal dependencies and market behavior.

3.5 Key Insights from EDA

- Gold prices show a clear long-term increasing trend.
- Open, High, Low, and Close prices have very strong positive correlations.
- Moving averages help smooth short-term market fluctuations and reveal long-term trends.
- Feature engineering improved the representation of market behavior for machine learning models.
- The dataset contains meaningful temporal patterns suitable for financial forecasting tasks.

4 Machine Learning Experiments

4.1 Train-Test Split

The dataset was divided into training and testing sets to evaluate model performance on unseen financial data.

Since gold price data represents time-series information, chronological order was preserved during splitting to avoid data leakage and unrealistic evaluation results.

The train-test splitting process used in this project is shown in Figure 6.

```
X = df[features]
y_price = df["Target_Next_Close"]
y_delta = df["Target_Delta"]
y_dir = df["Target_Direction"]

split_idx = int(len(df) * 0.8)
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]

y_train_price, y_test_price = y_price.iloc[:split_idx], y_price.iloc[split_idx:]
y_train_delta, y_test_delta = y_delta.iloc[:split_idx], y_delta.iloc[split_idx:]
y_train_dir, y_test_dir = y_dir.iloc[:split_idx], y_dir.iloc[split_idx:]

naive_price = df["Naive_Next_Close"].iloc[split_idx:]
naive_delta = df["Naive_Delta"].iloc[split_idx:]
naive_dir = df["Naive_Direction"].iloc[split_idx:]

tscv = TimeSeriesSplit(n_splits=5)

print(f"Train: {len(X_train)} row |  Test: {len(X_test)} row")
print(f"Test period: {df['Date'].iloc[split_idx].date()} - {df['Date'].iloc[-1].date()}")
```

Figure 6: Train-Test Split and Time-Series Preparation

The training dataset contained 4400 rows, while the testing dataset contained 1100 rows.

- Train set: 4400 rows
- Test set: 1100 rows

- Test period: 2021-09-15 — 2026-01-28

TimeSeriesSplit cross-validation was also used during hyperparameter tuning to preserve the temporal order of financial data.

4.2 Model Training and Hyperparameter Tuning

Several machine learning models were trained for both classification and regression tasks.

The following models were evaluated:

- Logistic Regression
- Random Forest
- Gradient Boosting
- Ridge Regression

GridSearchCV was used for hyperparameter optimization to improve model performance and identify the best parameter combinations.

The model training and optimization process is shown in Figures 7, 8, and 9.

Figure 7: Direction Prediction Models

Figure 8: Delta Prediction Models

Figure 9: Next Close Prediction Models

4.3 Direction Prediction Results

Classification models were trained to predict whether the gold price would increase or decrease in the next trading period.

The following evaluation metrics were used:

- Accuracy
- Precision
- F1-score

The best classification model based on the F1-score was Logistic Regression.

Model	Accuracy	Precision	F1-score
Naive Baseline	0.4709	0.5076	0.5076
Logistic Regression	0.5173	0.5427	0.5893
Random Forest	0.5355	0.5709	0.5576
Gradient Boosting	0.5136	0.5543	0.5167

Table 2: Direction Prediction Results

Logistic Regression achieved the highest F1-score, indicating better balance between precision and recall compared to other classification models.

4.4 Delta Prediction Results

Regression models were also trained to predict the difference between tomorrow's closing price and today's closing price.

The following evaluation metrics were used:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

Model	MAE	RMSE
Naive Baseline	18.7860	34.4593
Ridge Regression	18.9075	34.6985
Random Forest	53.5859	75.1161
Gradient Boosting	56.9594	78.8661

Table 3: Delta Prediction Results

The Naive Baseline achieved the lowest MAE score for delta prediction. This indicates that short-term price differences are highly difficult to predict accurately due to market volatility and noise.

4.5 Next Close Price Prediction Results

Regression models were trained to predict the next-day gold closing price.

The following evaluation metrics were used:

- MAE
- RMSE
- MAPE
- R-squared (R^2)

Model	MAE ($)	RMSE ($)	MAPE (%)	R^2
Naive Baseline	18.79	34.46	0.7438	0.9977
Ridge Regression	19.79	41.21	0.7666	0.9967
Random Forest	460.36	811.50	14.3476	-0.2839
Gradient Boosting	453.80	804.67	14.0941	-0.2624

Table 4: Next Close Price Prediction Results

The Naive Baseline achieved the best MAE and RMSE scores for next-day closing price prediction. Ridge Regression also demonstrated strong performance with very high R^2 values.

Random Forest and Gradient Boosting produced larger errors, suggesting possible overfitting and difficulties in generalizing future financial market behavior.

4.6 Feature Importance Analysis

Feature importance analysis was performed to identify which variables contributed most to prediction results.

The feature importance visualization is shown in Figure 10.

```
print_section("Conclusion")

print("Direction ]")
print(res_clf.to_string(index=False))

print("\n[Delta ($) ]")
print(res_del.to_string(index=False))

print("\n[Next Close ($) ]")
print(res_price.to_string(index=False))

print("\n═══════════════════════════════════════")
print(f"Best model for Direction    : {best_clf_name}")
print(f"Best model for Delta        : {best_del_name}")
print(f"Best model for Close  : {best_price_name}")
print("═══════════════════════════════════════\n")

print_section("Feature importance")

Tabnine | Edit | Test | Explain | Document
def print_feature_importance(model, feature_names, title, top_n=10):
    if hasattr(model, "feature_importances_"):
        imp = pd.Series(model.feature_importances_, index=feature_names)
        imp = imp.sort_values(ascending=False).head(top_n)
        print(f"\n{title}:")
        for feat, val in imp.items():
            print(f"  {feat:<22} {val:.4f}")

Tabnine | Edit | Test | Explain | Document
def get_estimator(pipeline_or_model):
    if hasattr(pipeline_or_model, "named_steps"):
        for key in ("clf", "reg"):
            if key in pipeline_or_model.named_steps:
                return pipeline_or_model.named_steps[key]
    return pipeline_or_model

print_feature_importance(get_estimator(best_rf_clf), features, "RF ─ Direction")
print_feature_importance(best_gb_clf, features, "GB ─ Direction")
print_feature_importance(get_estimator(best_rf_del), features, "RF ─ Delta")
print_feature_importance(best_gb_price, features, "GB ─ Next Close")
```

Figure 10: Feature Importance Analysis

The analysis revealed that lag-based variables and moving average features had the strongest influence on prediction performance. These features effectively captured temporal dependencies and historical market behavior.

4.7 Discussion

The machine learning experiments demonstrated that financial time-series forecasting is a highly challenging task due to market volatility and noise.

Classification models achieved moderate performance in predicting market direction, while regression models showed strong results for next-day closing price prediction.

The experiments also highlighted the importance of feature engineering, hyperparameter tuning, and time-series cross-validation when working with financial datasets.

5 Final Model

After evaluating multiple machine learning models, Ridge Regression and Logistic Regression demonstrated the most stable and reliable performance for gold price forecasting tasks.

For the classification task, Logistic Regression achieved the highest F1-score and showed better balance between precision and recall compared to other classification models.

For the regression task, Ridge Regression achieved very strong prediction performance with low error values and high R^2 scores. The model also demonstrated better generalization ability compared to ensemble models such as Random Forest and Gradient Boosting.

Although the Naive Baseline achieved slightly lower MAE values in some experiments, Ridge Regression was selected as the final machine learning model because it provided more stable predictions and represented a true predictive learning approach.

The final model was selected based on the following criteria:

- Prediction accuracy
- Stability on unseen data
- Generalization performance
- Interpretability
- Resistance to overfitting

The final model successfully captured important temporal patterns and relationships in historical gold price data.

6 Demo Description

An interactive demonstration application was developed to show how the trained machine learning model can be used for real-time gold price prediction.

The demo application allows users to input market-related features and receive predicted gold price outputs generated by the trained model.

The demonstration system includes the following steps:

- User inputs financial market values
- Preprocessing and feature engineering are applied

- The trained machine learning model generates predictions
- Predicted results are displayed to the user

The application demonstrates how machine learning can be applied to financial forecasting tasks in an interactive and user-friendly way.

The demo also provides practical understanding of how historical market data can be transformed into predictive financial insights using machine learning techniques.

7 Conclusions

This project explored the application of machine learning methods for gold price prediction using historical financial market data.

Several preprocessing techniques, feature engineering methods, and machine learning algorithms were evaluated through extensive experiments.

The results showed that machine learning models can effectively capture important market patterns and temporal relationships in financial time-series data.

Ridge Regression and Logistic Regression demonstrated the most stable and reliable performance among the tested models. The experiments also highlighted the importance of feature engineering, hyperparameter tuning, and time-series cross-validation in financial forecasting tasks.

Despite achieving good prediction performance, financial market forecasting remains highly challenging due to market volatility, economic uncertainty, and external factors that cannot be fully represented using historical price data alone.

Future improvements may include:

- Integration of macroeconomic indicators
- Use of deep learning models such as LSTM
- Incorporation of news and sentiment analysis
- Real-time financial data integration

Overall, this project demonstrated that machine learning provides valuable tools for financial forecasting and gold price prediction while also highlighting the complexity and uncertainty of real-world financial markets.